



操作系统实验报告

专 业： 人工智能

姓 名： 卢志强

学 号： 37220232203765

实验日期： 2025.12.4

指导教师： 曹冬林

目录

一、 实验目的：	3
二、 实验任务：	3
1. 任务一：实现调度算法	3
2. 任务二：测试	3
三、 实验步骤及实现方法	3
1. 实现调度算法	3
1.1 轮转调度 (Round Robin, RR)	3
1.2 实现优先级调度	4
1.3 实现先来先服务 (First-Come, First-Served, FCFS)	5
1.4 实现静态多级队列 (Static Multi-Level Queue, MLQ)	7
2. 测试	8
四、 实验结果分析	12
1. 基本参数个数测试：	12
2. 场景 1：混合负载 (IO+CPU)	12
3. 场景 2：护航效应	16
4. 场景 3：优先级饥饿	18
五、 总结	22

一、 实验目的：

通过实际编程实现和测试，深入理解不同进程调度算法的工作原理及其对系统性能的影响，掌握如何根据不同场景选择合适的调度策略。

二、 实验任务：

1. 任务一：实现调度算法

通过编程实现四种进程调度算法：轮转（RR）、优先级（Priority）、先来先服务（FCFS）、静态多级队列（Static Multi-level Queue），并在 `proc.c` 中完成相关代码。

2. 任务二：测试

测试方法在 `test_scheduler.c` 中进行：创建 6 个进程（包括 CPU 密集型和 I/O 密集型），记录所有进程的运行过程，并对比不同调度算法在以下四个指标上的结果：平均就绪时间、平均运行时间、平均休眠时间、平均周转时间。

三、 实验步骤及实现方法

1. 实现调度算法

1.1 轮转调度 (Round Robin, RR)

1.1.1 原理：公平性算法。系统为每个进程分配一个固定的时间片。当时间片用完时，如果进程还没结束，它会被强制挂起，放入就绪队列末尾，CPU 分配给下一个进程。

1.1.2 特点：响应时间好，适合交互式系统，防止某个进程长时间霸占 CPU。

1.1.3 实现步骤：

(1) 记录上次扫描的位置：如果每次调度器循环，都从 `ptable.proc[0]` 开始遍历。

假如 `ptable` 里有 10 个进程都是 `RUNNABLE` 的，每次发生时钟中断 `yield` 后，调度器又从头开始扫描。这意味着数组靠前的进程总是比靠后的更容易被选中（尤其是在高负载下），这在严格意义上不是完全公平的轮转。维护一个静态变量记录“上一次运行到了哪里”，下一次调度从这个位置接着往后找，而不是每次都从头找。

(2) 更新下次开始循环的位置。

```
struct proc *rrScheduler() {  
    struct proc *p;  
    static int last_idx = 0; // 记录上次扫描到的位置  
    // 从 last_idx 开始扫描一圈  
    for(int i = 0; i < NPROC; i++){  
        int idx = (last_idx + i) % NPROC; // 取模实现循环  
        p = &ptable.proc[idx];  
        if(p->state == RUNNABLE){  
            last_idx = (idx + 1) % NPROC; // 更新下次开始的位置  
            return p;  
        }  
    }  
    return 0;  
}
```

1.2 实现优先级调度

1.2.1 原理：

每个进程都有一个优先级属性。调度器总是选择当前就绪队列中优先级最高的进程运行。

1.2.2 特点:

重要的任务先执行。但要注意“饥饿”问题（低优先级进程可能永远得不到执行）。

1.2.3 实现步骤:

遍历所有的进程，比较处于运行态的各个进程之间的优先级，返回优先级最小的进程。

```
struct proc *priorityScheduler() {
    struct proc *p, *highP = 0;
    // 假设 PRIORITY 范围是 [1, 20]。初始化为高于最低优先级的值。
    int lowest_priority_seen = 21;

    // 注意: ptable.lock 已经在 scheduler() 中获取，所以这里不需

    // 遍历进程表寻找 RUNNABLE 且 priority 最小的进程
    for (p = ptable.proc; p < &ptable.proc[NPROC]; p++) {
        // 1. 必须是 RUNNABLE 状态
        if (p->state != RUNNABLE)
            continue;

        // 2. 查找 priority 字段中数值最小的进程（数值越小优先级越
        if (p->priority < lowest_priority_seen) {
            lowest_priority_seen = p->priority;
            highP = p;
        }
    }
}
```

1.3 实现先来先服务 (First-Come, First-Served, FCFS)

1.3.1 原理:

最简单的算法。按照进程进入就绪队列的先后顺序进行调度。一旦进程获得 CPU，它会一直运行直到结束或者因 I/O 请求而阻塞（非抢占式）。

1.3.2 特点:

实现简单，但对短作业不利（如果前面有个长作业，后面都要等），即“护航效应”。

1.3.3 实现步骤:

- (1) 遍历进程表，找到所有 RUNNABLE 的进程，选择其中 ctime（创建时间）最小的那个。
- (2) 增加 PID 作为二级调考虑到现代 CPU 速度很快，如果不加处理，批量创建的进程会有相同的 ctime。为了保证调度的确定性（Determinism），我增加了 PID 作为二级判断标准。pid 是单调递增的，PID 越小说明创建得越早。这确保了即使在高并发创建进程的场景下，算法依然严格遵守‘先来先服务’的语义。
- (3) 取消时钟中断：FCFS 算法是非抢占的，要求一旦进程获得 CPU，除非它自己结束或请求 I/O（睡眠），否则不应被时钟中断强制切换。

```
struct proc *fcfsScheduler() {  
    struct proc *p;  
    struct proc *minP = 0;  
    uint min_ctime = -1; // 初始化为最大无符号整数  
    // 遍历进程表  
    for(p = ptable.proc; p < &ptable.proc[NPROC]; p++){  
        if(p->state != RUNNABLE)  
            continue;  
        // 如果是第一个找到的，或者找到了创建时间更早的进程  
        // (如果 ctime 相同，保留原来的/PID 较小的，这里简单处理为找更小的)  
        if(minP == 0 || p->ctime < min_ctime || (p->ctime == min_ctime && p  
->pid < minP->pid)){  
            min_ctime = p->ctime;  
            minP = p;  
        }  
    }  
}
```

```
return minP;  
}
```

1.4 实现静态多级队列 (Static Multi-Level Queue, MLQ)

1.4.1 原理:

将就绪队列分成多个独立的队列（例如：高优先级队列和低优先级队列）。

1.4.2 特点:

通常采用固定优先级抢占调度。即：只要高优先级队列中有进程，就先调度高优先级的；只有高优先级队列为空时，才调度低优先级队列。

1.4.3 实现步骤:

- (1) 将进程基于优先级分为两个队列：高优先级队列（[1,10]）、低优先级队列（[11,20]）。
- (2) 高优先级队列内部采用 RR 调度算法，为每一个任务分配固定大小的时间片。

具体实现方式可以参考 1.2 节。

```
struct proc *p;  
static int last_idx = 0;  
  
for(int i = 0; i < NPROC; i++){  
    // 从上次停止的位置开始向后扫描（取模实现循环）  
    int idx = (last_idx + i) % NPROC;  
    p = &ptable.proc[idx];  
  
    // 如果是 RUNNABLE 且 优先级属于第一梯队（<=10）  
    if(p->state == RUNNABLE && p->priority <= 10){  
        // 更新下次开始的位置（+1）  
        last_idx = (idx + 1) % NPROC;  
        return p; // 找到即返回，实现 RR  
    }  
}  
  
struct proc *minP = 0;  
uint min_ctime = -1; // 辅助寻找最早创建的进程
```

- (3) 低优先级队列采用 FCFS 调度算法，优先创建的进程优先占用资源。优先扫

描并运行高优先级队列中的进程。只有当高优先级队列没有 `RUNNABLE` 进程时，才去运行低优先级队列的进程。

```
struct proc *minP = 0;
uint min_ctime = -1; // 辅助寻找最早创建的进程

for(p = ptable.proc; p < &ptable.proc[NPROC]; p++){
    // 如果是 RUNNABLE 且 优先级属于第二梯队 (>10)
    if(p->state == RUNNABLE && p->priority > 10){
        // FCFS 逻辑: 寻找 ctime (创建时间) 最小的进程
        // 增加 PID 作为二级判断, 处理 ctime 相同的情况
        if(minP == 0 || p->ctime < min_ctime || (p->ctime == min_ctime && p->pid < minP->pid)){
            min_ctime = p->ctime;
            minP = p;
        }
    }
}
```

2. 测试

2.1 核心常量设置：预定义调度算法 ID 和测试场景 ID。此处设置了三个测试场景分别为混合负载（CPU+IO）、护航效应（1 Long CPU + 5 short CPU）、优先级饥饿（High、Mid、Low）。

```
#define SCHED_DEFAULT 0
#define SCHED_PRIORITY 1
#define SCHED_FCFS 2
#define SCHED_RR 3
#define SCHED_SMLQ 4

// 测试场景定义
#define SCENE_MIX 1 // A: 混合负载 (CPU + IO)
#define SCENE_CONVOY 2 // B: 护航效应 (1 Long CPU + 5 Short CPU)
#define SCENE_PRIO 3 // C: 优先级饥饿 (High, Mid, Low)
```

2.2 解析参数并设置调度策略：

参数格式为：scheduler_test [调度算法 ID] [测试场景 ID]

当参数解析失败后为用户提供以下提示：

Usage: scheduler_test <algo_id> <scene_id>

Algo ID: 0=Default, 1=Priority, 2=FCFS, 3=RR, 4=SMLQ

Scene ID: 1=Mix(CPU+IO), 2=Convoy(Long+Short), 3=Prio(High/Mid/Low)

```
if(argc < 3){
    printf(1, "Usage: scheduler_test <algo_id> <scene_id>\n");
    printf(1, "Algo ID: 0=Default, 1=Priority, 2=FCFS, 3=RR, 4=SMLQ\n");
    printf(1, "Scene ID: 1=Mix(CPU+IO), 2=Convoy(Long+Short), 3=Prio(High/Mid/Low)\n");
    exit();
}

algo_id = atoi(argv[1]);
scene_id = atoi(argv[2]);

// 1. 设置调度策略
printf(1, "\n[INFO] Setting Scheduler Policy to ID: %d\n", algo_id);
if (setscheduler(algo_id) < 0) {
    printf(1, "[ERROR] setscheduler failed! (Check if syscall is implemented)\n");
    exit();
}
```

2.3 实现 CPU 密集任务：

参数 `scale` 表示相对于基础任务的倍率（`scale = 5`，相当于 5 倍的基础工作量），实现在不同场景中使用不同计算任务量。可能通过多层空循环实现数据累加运算消耗 CPU 资源，无 IO 操作，模拟计算密集任务。

```
void work_cpu(int scale) {
    int i, j, k_loop;
    volatile int k = 0;
    for (i = 0; i < scale; i++) {
        for(k_loop = 0; k_loop < 30; k_loop++) {
            for (j = 0; j < 500000; j++) {
                k = k * j + 1;
            }
        }
    }
}
```

2.4 实现 IO 密集任务：

通过 `sleep` 操作让进程主动放弃 CPU 资源，模拟网络请求、磁盘读写等 IO 任务。

```

// IO 密集型任务
// 频繁休眠，少量计算
void work_io(int scale) {
    int i, j;
    int k = 0;
    for (i = 0; i < scale * 10; i++) { // 循环更多次，但每次很短
        // 做一点点计算
        for (j = 0; j < 1000000; j++) {
            k++;
        }
        // 然后睡觉（模拟 IO 等待）
        sleep(10);
    }
}

```

2.5 场景 1：混合负载（IO+CPU）

- (1) 实现：创建 6 个子进程，进程号位偶数的执行 `work_cpu`，优先级设置为 5；
进程号为奇数的执行 `work_io`，优先级设置为 15。IO 和 CPU 的倍率 `scale` 均设置为 1。

PID	4	5	6	7	8	9
任务类型	CPU	CPU	CPU	IO	IO	IO
优先级	5	5	5	15	15	15

表一. 不同进程的任务类型以及优先级设置

- (2) 目的：测试调度器在面对“计算型”和“交互型”任务混杂时，是否能保证交互型任务（IO）的响应速度（即 RR 算法的优势）。

```
// --- 场景 A: 混合负载 (Mix) ---
if (scene_id == SCENE_MIX) {
    proc_count = 6;
    for (i = 0; i < proc_count; i++) {
        pid = fork();
        if (pid == 0) {
            // 子进程
            if (i % 2 == 0) {
                work_cpu(1);
            } else {
                work_io(1);
            }
            setpriority(current_pid, i % 2 == 0 ? 5 : 15);
            exit(); // 这里的 exit 实际上在 work 函数里也会调用，作为双重保险
        }
    }
}
```

2.6 场景 2：护航效应

- (1) 实现：先创建一个超长的 CPU 任务（大象进程），然后 `sleep(10)`，确保长任务已经成功进入了就绪队列。然后再创建五个短的 CPU 任务。

```
pid = fork();
if (pid < 0) printf(1, "[ERROR] Fork failed for Long Task\n");
else if (pid == 0) { work_cpu(20); exit(); }
else {
    proc_count++;
    sleep(10); // 让长任务先跑
    // 创建短任务
    for (i = 0; i < 5; i++) {
        pid = fork();
        if (pid < 0) { printf(1, "[ERROR] Fork failed for Short Task\n"); break; }
        if (pid == 0) { work_cpu(2); exit(); }
        proc_count++;
    }
}
```

- (2) 目的：验证当有一个长作业时，短时任务是否会被严重阻塞。

2.7 场景 3：优先级饥饿

- (1) 实现：创建 4 个高优先级的 IO 密集型任务，再创建两个低优先级的 CPU 密集型任务。各个进程的设置如表一所示。
- (2) 目的：验证高优先级是否真的先跑，以及低优先级是否会被“饿死”。

PID	4	5	6	7	8	9
任务类型	IO	IO	IO	IO	CPU	CPU
优先级	5	5	8	8	15	15

表二. 不同进程的任务类型以及优先级设置

```

if (pid < 0) { printf(1, "[ERROR] Fork failed!\n"); break; }
if (pid == 0) {
    int prio;
    if (i < 2) {
        prio = 15;
        setpriority(getpid(), prio);
        work_cpu(1);
    } // High
    else if (i < 4) {
        prio = 5;
        setpriority(getpid(), prio);
        work_cpu(10);
    }
    else {
        prio = 8;
        setpriority(getpid(), prio);
        work_cpu(10);
    }
    exit();
}

```

四、实验结果分析

1. 基本参数个数测试:

(1) 参数数量不足:

```

$ scheduler_test 0
Usage: scheduler_test <algo_id> <scene_id>
Algo ID: 0=Default, 1=Priority, 2=FCFS, 3=RR, 4=SMLQ
Scene ID: 1=Mix(CPU+IO), 2=Convoy(Long+Short), 3=Prio(High/Mid/Low)
$

```

(2) 调度算法 ID 出错:

```

$ scheduler_test 8 1
[ERROR] setscheduler failed!
Algo ID: 0=Default, 1=Priority, 2=FCFS, 3=RR, 4=SMLQ
$

```

2. 场景 1: 混合负载 (IO+CPU)

(1) 如图一所示, 创建了 6 个子进程, 其中 PID 为 4,6,8 的子进程执行 `work_cpu()` 任务, 优先级设置为 15; PID 为 5,7,9 的子进程执行 `work_io()` 任务, 优先级设置 5。各个进程的执行时间如图一所示。

a. Priority	PID	Type	Ready	Run	Sleep	Total
	4	CPU	6	8	0	14
	6	CPU	11	6	0	17
	8	CPU	18	15	0	33
	5	IO	8	22	14	44
	7	IO	22	8	14	44
	9	IO	27	2	14	43
b. FCFS	PID	Type	Ready	Run	Sleep	Total
	6	CPU	1	2	0	3
	8	CPU	2	3	0	5
	4	CPU	0	16	0	16
	5	IO	9	4	13	26
	7	IO	14	0	14	28
	9	IO	14	2	14	30
c. RR	PID	Type	Ready	Run	Sleep	Total
	8	CPU	21	5	0	26
	4	CPU	15	19	0	34
	6	CPU	20	18	0	38
	9	IO	33	4	16	53
	7	IO	26	11	16	53
	5	IO	30	16	16	62

图一：不同调度算法下各个子进程执行的具体情况

419: RUN -> PID 5 (Pri:5)	406: RUN -> PID 4 (Pri:10)	[INFO] Starting Scenario 1.
420: RUN -> PID 5 (Pri:5)	407: RUN -> PID 5 (Pri:10)	[INFO] Starting Algo 2...
421: RUN -> PID 5 (Pri:5)	407: RUN -> PID 6 (Pri:10)	503: RUN -> PID 4 (Pri:10)
422: RUN -> PID 5 (Pri:5)	409: RUN -> PID 3 (Pri:10)	506: RUN -> PID 5 (Pri:10)
423: RUN -> PID 5 (Pri:5)	409: RUN -> PID 5 (Pri:10)	506: RUN -> PID 6 (Pri:10)
423: RUN -> PID 9 (Pri:5)	409: RUN -> PID 7 (Pri:10)	508: RUN -> PID 5 (Pri:10)
423: RUN -> PID 8 (Pri:15)	409: RUN -> PID 8 (Pri:10)	508: RUN -> PID 7 (Pri:10)
423: RUN -> PID 7 (Pri:5)	412: RUN -> PID 3 (Pri:10)	509: RUN -> PID 5 (Pri:10)
424: RUN -> PID 7 (Pri:5)	412: RUN -> PID 5 (Pri:10)	509: RUN -> PID 7 (Pri:10)
424: RUN -> PID 7 (Pri:5)	412: RUN -> PID 7 (Pri:10)	510: RUN -> PID 5 (Pri:10)
424: RUN -> PID 7 (Pri:5)	412: RUN -> PID 9 (Pri:10)	511: RUN -> PID 7 (Pri:10)
424: RUN -> PID 7 (Pri:5)	414: RUN -> PID 5 (Pri:10)	512: RUN -> PID 5 (Pri:10)
424: RUN -> PID 7 (Pri:5)	414: RUN -> PID 7 (Pri:10)	512: RUN -> PID 7 (Pri:10)
424: RUN -> PID 7 (Pri:5)	414: RUN -> PID 9 (Pri:10)	513: RUN -> PID 5 (Pri:10)
425: RUN -> PID 5 (Pri:5)	416: RUN -> PID 5 (Pri:10)	513: RUN -> PID 7 (Pri:10)
425: RUN -> PID 9 (Pri:5)	416: RUN -> PID 7 (Pri:10)	514: RUN -> PID 5 (Pri:10)
425: RUN -> PID 8 (Pri:15)	416: RUN -> PID 9 (Pri:10)	514: RUN -> PID 7 (Pri:10)
426: RUN -> PID 5 (Pri:5)	418: RUN -> PID 5 (Pri:10)	514: RUN -> PID 8 (Pri:10)
427: RUN -> PID 5 (Pri:5)	418: RUN -> PID 7 (Pri:10)	517: RUN -> PID 5 (Pri:10)
427: RUN -> PID 7 (Pri:5)	418: RUN -> PID 9 (Pri:10)	517: RUN -> PID 7 (Pri:10)
427: RUN -> PID 8 (Pri:15)	420: RUN -> PID 5 (Pri:10)	518: RUN -> PID 5 (Pri:10)
428: RUN -> PID 5 (Pri:5)	422: RUN -> PID 3 (Pri:10)	518: RUN -> PID 7 (Pri:10)
429: RUN -> PID 5 (Pri:5)	422: RUN -> PID 7 (Pri:10)	519: RUN -> PID 5 (Pri:10)
430: RUN -> PID 5 (Pri:5)	423: RUN -> PID 5 (Pri:10)	519: RUN -> PID 7 (Pri:10)
430: RUN -> PID 7 (Pri:5)	423: RUN -> PID 7 (Pri:10)	520: RUN -> PID 5 (Pri:10)
431: RUN -> PID 5 (Pri:5)	423: RUN -> PID 9 (Pri:10)	520: RUN -> PID 7 (Pri:10)
432: RUN -> PID 5 (Pri:5)	424: RUN -> PID 5 (Pri:10)	521: RUN -> PID 5 (Pri:10)
432: RUN -> PID 7 (Pri:5)	424: RUN -> PID 7 (Pri:10)	521: RUN -> PID 7 (Pri:10)
		522: RUN -> PID 5 (Pri:10)
		524: RUN -> PID 5 (Pri:10)

a. Priority

b. FCFS

c. RR

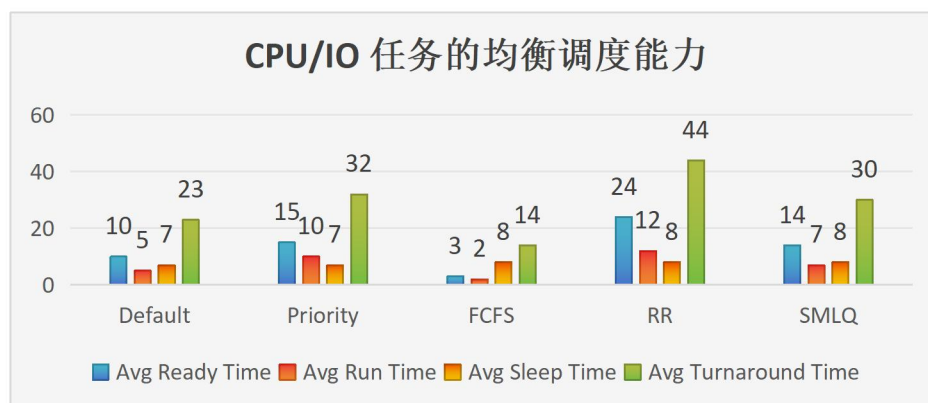
图二：不同调度算法运行时子进程调换的日志信息

5	7	9	5	7	9	5	7				4	6	8
a.Priority													
4	5				6	7			8	9			
b. FCFS													
4	5	6	7	5	7	5	7				9	9	9
c. RR													

图三：不同调度算法运行时子进程的执行状态

(2) 结合图二和图三，可以基本验证几种调度算法实现的正确性。

- ① 在图二 a 和图三 a 中可以看出，虽然优先创建的 PID=4 的进程，但是由于 IO 任务的进程 5,7,9 的优先级更高，所以优先执行 PID=5 的子进程。由于基于 Priority 的调度算法是**抢占性**的，所以会产生时钟中断，PID=5,7,9 的进程交替执行。等 IO 任务的子进程执行完毕之后 PID = 4,6,8 的 CPU 密集型任务才开始执行。
- ② 在图二 b 和图三 b 中可以看出，在 FCFS 调度算法下，各个子进程之间是按照创建进程的时间进行计算的。首先执行 PID=4 的 CPU 密集型任务，执行完毕之后再执行 PID=5 的 IO 密集型任务。同时，FCFS 调度算法是非抢占性的，不会产生时钟中断。
- ③ 在图二 c 和图三 c 中可以看出，进程按时间片轮流进入队列，RR 为每个进程分配固定时间片，时间片用完后进程重新排队，因此短任务会被快速处理，长任务则多次排队后集中完成。RR 的优点是公平性好，短任务响应快，适合交互式场景，但是长任务周转时间长，且频繁切换带来额外开销，图中高优先级长任务 9 被多次延迟，直到短任务处理完才集中执行。



图五. CPU/IO 任务的均衡调度能力

(3) 场景 1 是 “混合负载（3 个 CPU 密集 + 3 个 IO 密集进程）”，需重点关注 “CPU/IO 任务的均衡调度能力” “响应性（就绪时间）” “整体效率（周转时间）” 三个维度。结合图五，以下是对各算法的分析：

按 “平均周转时间（总效率）” 从优到差：

FCFS（14）> Default（23）> SMLQ（30）> Priority（32）> RR（44）

① **FCFS（先来先服务）**：整体效率最优，但隐藏风险

1) 优点：平均就绪时间（3）全场最低：进程创建后几乎无需等待，直接获得 CPU；平均周转时间（14）全场最优：混合负载下总耗时最短。

2) 缺点：平均运行时间异常低：结合混合场景，大概率是 CPU 进程被 IO 进程阻塞（FCFS 按顺序调度，若 IO 进程先到达，CPU 进程会被长时间阻塞，导致 CPU 进程的 “运行时间” 被压缩）；

② **RR（时间片轮转）**：表现最差，混合场景下不适用。RR 的核心是 “频繁时间片切换”，在混合负载下对 CPU 密集进程而言，切换开销大，导致运行时间增加；对 IO 密集进程，时间片未用完就进入睡眠，浪费 CPU 资源，最终整体周转时间剧增。

③ **Priority（优先级调度）**：效率差，对混合负载不友好。就绪时间（15）、运行时间（10）、周转时间（32）均较差；睡眠时间（7）虽低，但被高就绪 / 运行时间拖累。

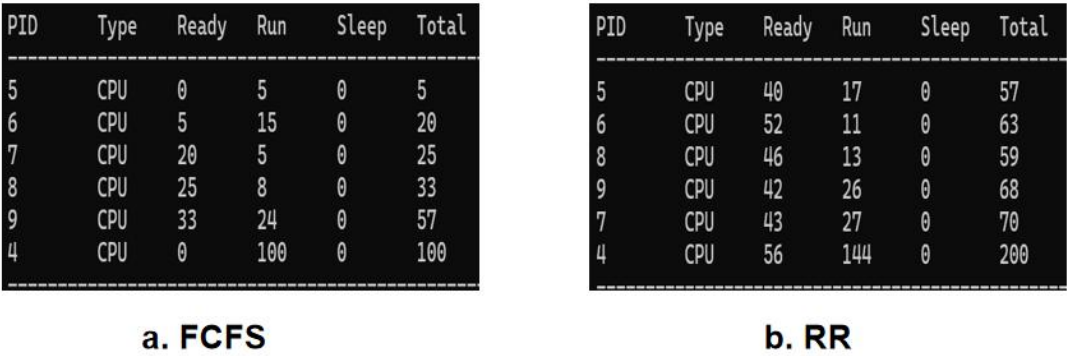
④ **SMLQ（静态多级队列）**：高优先级 IO 密集型进程通过 RR 轮转，平均就绪时间和响应速度优于基于优先级调度的算法，低优先级 CPU 密集型进程虽仍需等待高队列空闲，但高队列内部无长期独占问题，整体周

转时间相比于基于优先级调度的算法更优。

3. 场景 2：护航效应

	Default	Priority	FCFS	RR	SMLQ
Avg Ready Time	21	13	46	25	25
Avg Run Time	25	26	39	26	40
Avg Sleep Time	0	0	0	0	0
Avg Turnaround Time	47	40	86	52	65

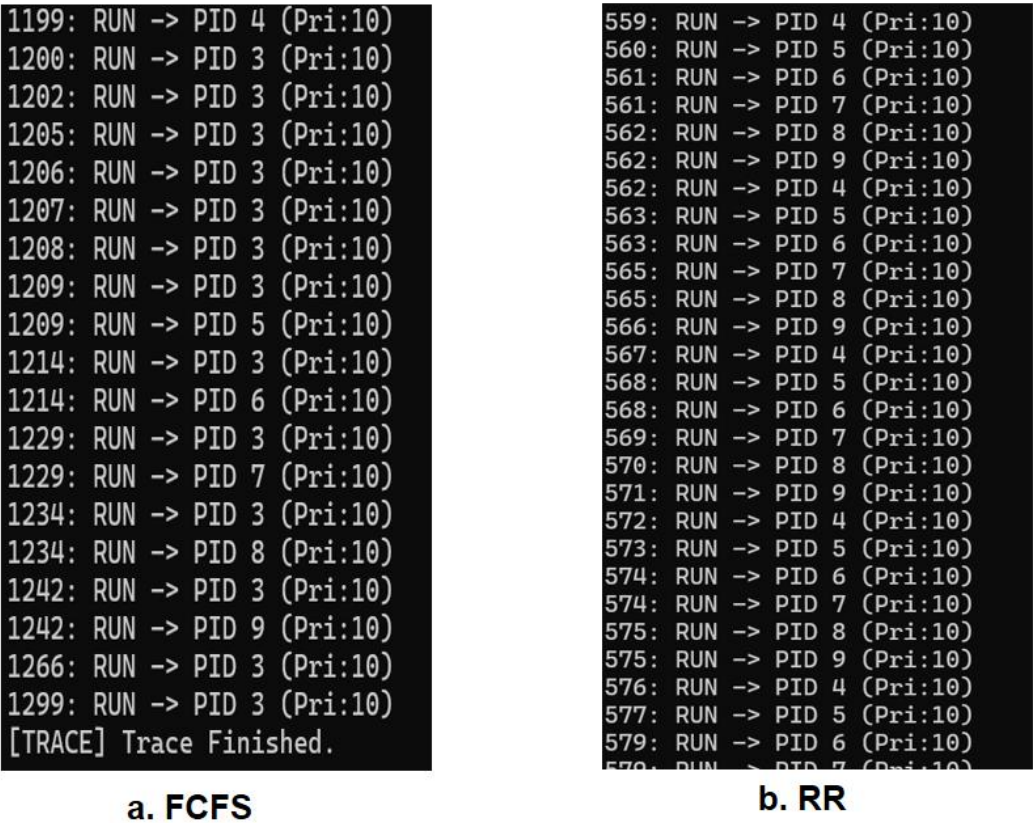
表二. 场景 2 下不同调度算法的性能表现



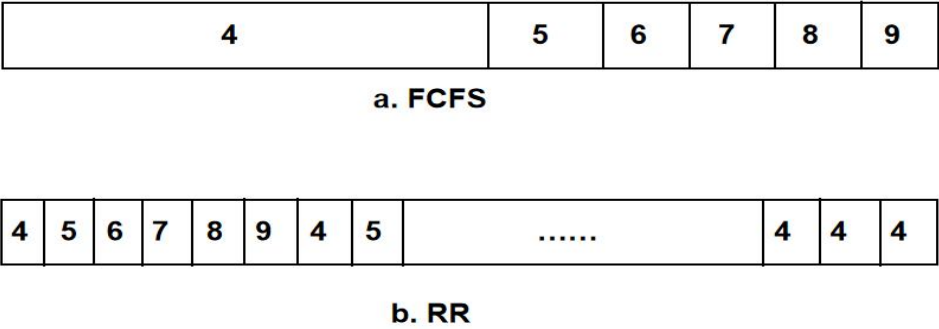
图六. FCFS 调度算法和 RR 调度算法下各个任务的性能表现

- (1) **FCFS（先来先服务）**：由图七 a 和图八 a 可知，FCFS（先来先服务）的执行顺序是长任务 4 占据队列前端的大段空间，短任务 5/6/7/8/9 依次排在其后。FCFS 严格按“任务到达顺序”调度，长任务 4 先到达并独占 CPU，所有短任务必须等待长任务完全执行完毕后，才能依次被调度。这正是“护航效应”的典型表现——长任务阻塞了所有短任务，导致短任务的就绪时间 / 周转时间急剧增加，整体响应性极差。
- (2) **RR（时间片轮转）**：由图七 b 和图八 b 可知，RR（时间片轮转）的执行顺序是长任务 4 与短任务 5/6/7/8/9 交替出现，且长任务 4 在队列末尾集中重复。RR 为每个任务分配固定时间片，长任务 4 和短任务 5/6/7/8/9 按到达顺序轮

流占用 CPU（时间片用完后重新排队）。短任务因执行时间短，在 1-2 个时间片内即可完成，因此会快速从队列中消失。长任务 4 需要多个时间片才能完成，因此会多次重新排队，最终在短任务全部完成后，集中消耗剩余时间片。短任务无需等待长任务完成，能快速获得 CPU 时间片，由表二可以看出,RR 有效缓解了护航效应,短任务的响应性和周转时间会显著优于 FCFS。



图七. FCFS 调度算法和 RR 调度算法各个任务的执行日志



图八. FCFS 调度算法和 RR 调度算法各个任务的执行顺序

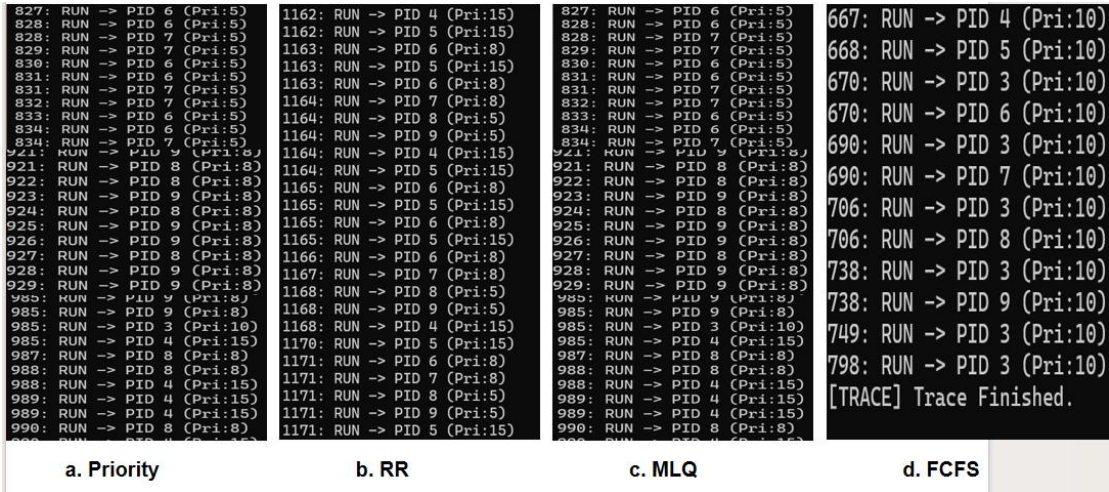
在场景 2（护航效应场景）下，FCFS 会放大长任务的阻塞问题，而 RR 能通过时间片轮转保障短任务的响应性，是更适配场景 2 的调度算法。

4. 场景 3：优先级饥饿

	Default	Priority	FCFS	RR	SMLQ
Avg Ready Time	48	98	25	53	102
Avg Run Time	21	50	26	25	48
Avg Sleep Time	32	0	0	30	0
Avg Turnaround Time	101	140	52	109	154

表三. 场景 3 下不同调度算法的性能表现

(1) 基于优先级的调度：高优先级进程 6/7/8/9 占据队列前端，低优先级 4/5 仅在末尾出现，说明高优先级进程被持续调度，低优先级进程被延后。优先级调度优先保障高优先级进程，但低优先级进程出现**严重饥饿**（就绪时间极长），导致整体平均指标拉垮 —— 这是无“优先级老化”机制的典型问题（高优先级进程长期占用 CPU，低优先级进程无法获得调度）。结合图十一可以看出大量的时间都处于就绪队列中。



图九. 不同调度算法各个任务的执行日志

6	7	6	7	8	9	8	9		4	5	4	5
---	---	---	---	---	---	---	---	-------	---	---	---	---

a. Priority

4	5	6	7	8	9	4	5		6	7	8	9
---	---	---	---	---	---	---	---	-------	---	---	---	---

b. RR

6	7	8	9	6	7	8	9		4	5
---	---	---	---	---	---	---	---	-------	---	---

c. MLQ

4	5	6	7	8	9
---	---	---	---	---	---

d. FCFS

图十. 不同调度算法各个任务的执行顺序

- (2) **RR（时间片轮转）**：高 / 低优先级进程（4/5/6/7/8/9）交替出现，低优先级 4/5 未被排挤，高优先级 6/7/8/9 在末尾集中执行。平均就绪时间（53），RR 按 “时间片轮流” 调度，完全平等对待所有进程—— 无优先级区分，因此无饥饿问题，但高优先级进程的响应性无法保障（高优先级进程与低优先级进程共享时间片）。

PID	Type	Ready	Run	Sleep	Total
7	CPU	10	83	0	93
6	CPU	31	64	0	95
9	CPU	110	53	0	163
4	CPU	164	11	0	175
5	CPU	172	10	0	182
8	CPU	101	81	0	182

a. Priority

PID	Type	Ready	Run	Sleep	Total
4	CPU	27	36	0	63
7	IO	45	11	11	67
5	CPU	46	37	0	83
6	IO	55	22	12	89
9	IO	78	18	78	174
8	IO	69	29	80	178

b. RR

PID	Type	Ready	Run	Sleep	Total
7	CPU	132	105	0	237
6	CPU	113	124	0	237
9	CPU	127	110	0	237
4	CPU	237	11	0	248
5	CPU	249	12	0	261
8	CPU	169	104	0	273

c. MLQ

PID	Type	Ready	Run	Sleep	Total
5	CPU	3	2	0	5
4	CPU	2	23	0	25
6	CPU	5	36	0	41
7	CPU	25	48	0	73
8	CPU	38	43	0	81
9	CPU	70	60	0	130

d. FCFS

图十一. 不同调度算法各个任务的具体执行情况

- (3) SMLQ（多级反馈队列）：高队列 RR 特性缓解了同优先级进程饥饿，高优先级 IO 进程通过 RR 轮转，无同队列饥饿问题，平均运行时间（低于 Priority 的 50）更优；因为高队列优先，低优先级 CPU 进程虽仍存在饥饿。值得注意的是，SMLQ（静态多级队列）在场景 3 中表现最差（平均周转时间 154）。这是因为 SMLQ 结合了‘优先级压制’和‘队列内轮转’的特性。在高优先级队列（前台）未执行完之前，低优先级任务完全被饿死（Ready Time 极高）；而高优先级队列内部采用 RR 调度，又引入了切换开销。这种‘双重打击’导致在资源竞争激烈的场景下，若无老化机制，SMLQ 的整体效率可能会低于简单的 FCFS 或 Priority。
- (4) FCFS 按“到达顺序”调度，完全忽略优先级——高、低优先级进程按到达顺序依次执行，因此无饥饿问题，整体效率最高，但高优先级进程的响应性无法保障（高优先级进程可能被低优先级进程阻塞）。

5. 实验的优点：

- (1) **设计完整性与场景覆盖全面**：实验围绕进程调度核心目标，完整实现了轮转（RR）、优先级（Priority）、先来先服务（FCFS）、静态多级队列（SMLQ）四种经典调度算法，覆盖了抢占式与非抢占式、优先级感知与公平性导向等不同调度逻辑。测试场景设计针对性极强，混合负载、护航效应、优先级饥饿三大场景分别对应实际系统中“任务类型混杂”“长短任务并存”“优先级差异显著”的典型问题，能够全面验证不同算法的适配性。
- (2) **实现细节严谨，贴合底层逻辑**：算法实现过程中注重细节优化与语义准确性。例如 RR 算法通过维护 `last_idx` 变量实现循环扫描，避免了数组靠前进程的调度偏向；FCFS 算法引入 PID 作为二级判断标准，解决了高并发场景下进程创建时间相同的调度确定性问题；优先级调度明确了优先级数值与优先级高低的映射关系，同时规避了进程表锁的重复获取问题，体现了对操作系统底层调度机制的深刻理解。
- (3) **测试方案科学，指标体系完善**：测试环节通过创建 CPU 密集型与 I/O 密集型两类进程，模拟真实系统负载；定义平均就绪时间、平均运行时间、平均休眠时间、平均周转时间四大核心指标，从响应性、执行效率、资源利用等维度全面量化算法性能。同时通过进程执行日志、调度顺序可视化等方式，直观呈现算法运行过程，便于定性分析与问题定位。

6. 实验的不足：

- (1) **算法功能存在局限性**：部分算法缺乏实际系统所需的优化机制。优先级调度与 SMLQ 算法未实现“优先级老化”功能，导致低优先级进程出现严重饥饿问题，与真实操作系统中“兼顾优先级与公平性”的设计目标

存在差距；RR 算法的时间片长度为固定值，未考虑进程类型（CPU 密集 / I/O 密集）动态调整时间片，影响了不同负载下的适配性；SMLQ 算法仅划分两级队列，且队列调度策略与优先级调度高度重合，未体现多级队列的灵活性优势。

(2) **测试场景与参数设置不够灵活**：测试场景中进程数量固定为 6 个，未考虑不同系统负载强度（如高并发、低负载）对算法性能的影响；CPU 密集型与 I/O 密集型任务的工作量通过固定倍率设置，未提供参数化配置接口，难以模拟不同复杂度的任务场景；时间片长度、优先级范围等核心参数在代码中硬编码，缺乏可配置性，不便于开展多参数组合测试。

(3) **性能分析存在片面性**：实验分析未考虑调度算法的额外开销。例如 RR 算法的频繁进程切换会产生上下文切换开销，但实验中未量化该开销对系统性能的影响；SMLQ 算法的队列切换逻辑也存在一定开销，未在指标中体现。此外，分析过程未涉及算法的稳定性测试，如长时间运行后是否存在进程状态异常、调度死锁等问题。

五、 总结

本次操作系统实验围绕进程调度算法的实现与测试展开，通过编程实现四种经典调度算法、设计针对性测试场景、量化分析性能指标，深入理解了进程调度的核心原理与实际应用场景。

实验结果表明，不同调度算法各具特性，适配不同的系统需求：FCFS 算法实现简单、开销小，在混合负载场景中表现出优异的整体效率，但在长短任务并存时会产生严重的护航效应，适合任务类型单一、对响应性要求不高的场景；RR 算法通过时间片轮转机制保障了进程调度的公平性，有效缓解了护航效应，适合

交互式系统，但频繁的上下文切换导致整体周转时间较长，且对 CPU 密集型任务不友好；优先级调度与 SMLQ 算法能够优先保障高优先级进程的执行，适合对任务优先级敏感的场景，但缺乏老化机制导致低优先级进程饥饿，难以兼顾公平性。

通过本次实验，不仅掌握了进程调度算法的编程实现方法，理解了调度算法对系统性能的影响机制，还深刻认识到实际操作系统调度器的设计复杂性——需要在公平性、响应性、效率等多维度之间寻求平衡。同时，实验中暴露的算法局限性、测试灵活性不足等问题，也为后续学习指明了方向。未来可进一步优化算法设计，添加优先级老化、动态时间片调整等功能；完善测试方案，增加负载强度、任务复杂度等变量维度；深化性能分析，量化调度开销对系统性能的影响，从而更全面地模拟真实操作系统的调度机制，提升对操作系统核心原理的理解与实践能力。